코딩 101

언어 종류부터 기초 · 중급 · 고급 · 고수까지 — docs/coding 요약

기초부터 고수까지 6 단계

1	2	3	4	5	6
준비	기초	중급	실전	고급	고수
코딩이란 · 언어 지도	변수 · 조건 · 반복 · 함수 · 파일	객체지향 · 자료구조 · 도구	웹 개발 · SQL · API	메모리 · 동시성 · 네트워크	설계 · 리뷰 · 시스템 · AI
1–3 장	4–5 장	6–8 장	9 장	10 장	11–12 장

읽기만 하지 말고 직접 쳐 보기 · 오류는 정상 · 작은 걸 끝까지 만들기

코딩 = 컴퓨터에게 주는 아주 자세한 요리법

1

CPU

요리사

명령을 하나씩 실행

2

메모리 (RAM)

조리대

작업 중인 데이터 .
끄면 사라짐

3

저장장치

냉장고 · 창고

파일을 오래 보관

4

입출력

주문서 · 서빙

키보드 · 화면 ·
네트워크

컴퓨터는 0 과 1(비트) 만 안다 . 사람이 읽기 쉬운 언어로 쓰고 , 번역해서 실행한다

번역 방법 3 가지

컴파일

책을 미리 통째로 번역

C · C++ · Go · 러스트

실행이 빠름 , 실행 전 오류 발견

인터프리터

동시통역사

파이썬 · JS · 루비 · PHP

바로 실행 , 배우기 쉬움

가상 머신

공통어로 번역 후 현지 통역

자바 · C# · 코틀린

여러 운영체제에서 같은 코드

저수준 ↔ 고수준 언어

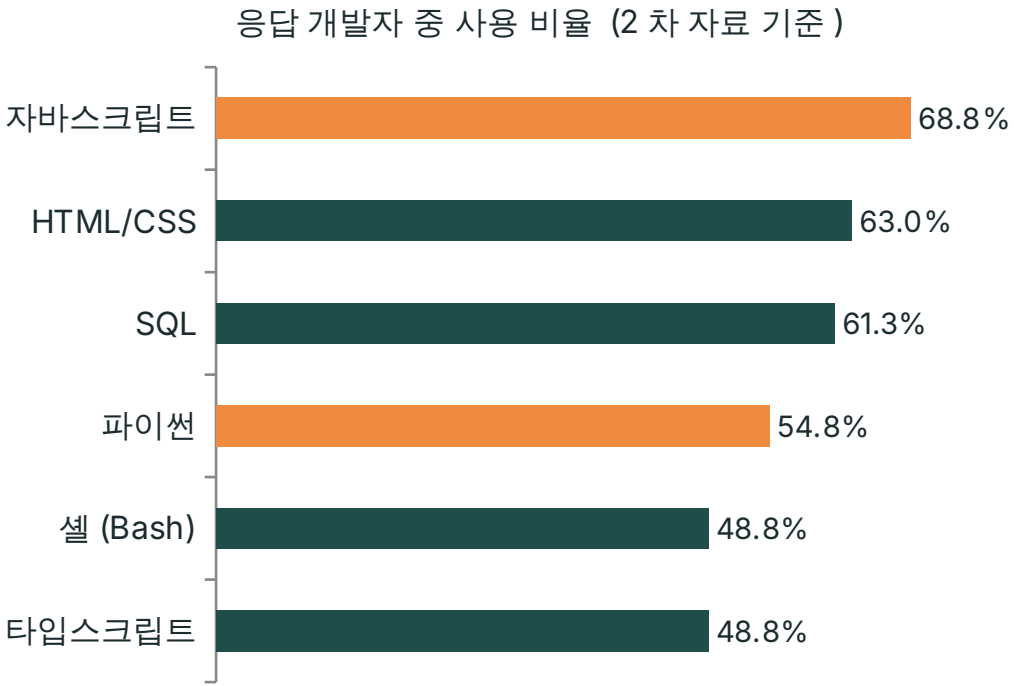
기계어	어셈블리	C · C++ · 러스트	자바 · C# · Go	파이썬 · JS
0 과 1	MOV AL, 61h	메모리 직접 관리	자동 메모리 관리	사람 말에 가까움

← 빠르고 기계를 세밀하게 제어 · 어렵고 실수하기 쉬움

쉽고 빨리 만듦 · 상대적으로 느림 →

요즘은 경계가 흐려짐 : 자바스크립트도 실행 중에 기계어로 바꿔 (JIT) 꽤 빠르게 돌린다

많이 쓰는 언어 (2025 개발자 설문)



TIOBE 2026 년 10 월

1 위 파이썬

2 위 C

3·4 위 C++ · 자바

순위 ≠ 내가 배울 언어 . 만들고 싶은 것부터 정하기

무엇을 만들고 싶은가 → 언어

1 웹 화면

HTML/CSS · JS · TS

2 웹 서버

자바 · 파이썬 · Node.js · Go · PHP

3 모바일 앱

코틀린 · 스위프트 · Flutter

4 게임

C++(언리얼) · C#(유니티)

5 데이터 · AI

파이썬 · R · SQL

6 시스템 · 임베디드

C · C++ · 러스트

C · C++ · C#: 이름은 비슷, 성격은 다르다

C

1970 년대, 언어들의 할아버지

- 운영체제 · 임베디드 칩
- 매우 빠름
- 메모리를 직접 관리 → 실수 위험

난이도 ★★★★

C++ (씨쁠쁠)

C + 객체지향 + 많은 기능

- 게임 엔진 (언리얼) · 브라우저
- C 만큼 빠름
- 기능이 많아 배우기 어려움

난이도 ★★★★★

C# (씨샵)

마이크로소프트, 자바와 닮음

- 유니티 게임 · 윈도우 · .NET 서버
- 깔끔한 문법
- 메모리 자동 관리

난이도 ★★★

같은 코드 , 다른 말투

```python
def total(n):
    s = 0
    for i in range(1, n+1):
        s += i
    return s

print(total(10))
```

파이썬

```c
int total(int n) {
  int s = 0;
  for (int i=1; i<=n; i++)
    s += i;
  return s;
}
// printf("%d", total(10));
```

C

```javascript
function total(n) {
  let s = 0;
  for (let i=1; i<=n; i++)
    s += i;
  return s;
}
console.log(total(10));
```

자바스크립트

생각 (알고리즘) 은 같고 말투 (문법) 만 다르다 — 10 개 언어 비교는 3 장 (모두 실행 확인)

모든 언어의 4 가지 뼈대

1 변수

이름표 붙은 상자

```
age = 20
```

2 조건문

갈림길

```
if score >= 90:
```

3 반복문

컨베이어

```
for i in range(5):
```

4 함수

레시피 카드

```
def add(a, b):
```

초보자 실수 1 위 "10"(글자) ≠ 10(숫자) · = (넣기) ≠ == (비교) · range(1, 5) 는 1~4

데이터 묶음 : 리스트와 딕셔너리

```
fruits = ["사과", "포도"]
fruits[0]        # 사과
fruits.append("딸기")
```

```
person = {"name": "민수",
          "age": 20}
person["name"]   # 민수
```

리스트 : 번호로 찾기 , 번호는 0 부터

딕셔너리 : 이름 (키) 으로 찾기

둘을 섞으면 = 실제 데이터 모양 (JSON) [{"name": "쿠키", "price": 3000}, ...]

오류는 친절한 편지 : 맨 아래 줄부터 읽기

```
Traceback (most recent call last):
  File "test.py", line 3
    print(10 / 0)
ZeroDivisionError: division by zero
```

오류	흔한 원인
SyntaxError	괄호 · 따옴표 · 콜론 빠짐
NameError	변수 이름 오타
TypeError	"10" + 5
IndexError	번호 범위 밖

고치는 순서

1. 맨 아래 줄 읽기
2. 줄 번호로 가기
3. print 로 값 확인
4. 오류 문장 검색 ·AI 질문

객체지향 : 붕어빵 틀과 붕어빵

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(self.name, "멍멍!")

choco = Dog("초코")
choco.bark()
```

클래스	틀 (설계도)
객체	찍어 낸 붕어빵
속성	가진 데이터
메서드	할 수 있는 행동

4 가지 핵심 캡슐화 · 상속 · 다형성 · 추상화 (상속보다 조합을 더 자주)

빅오 : 데이터 100 만 개일 때 일하는 양

O(1)	1 번	사물함 번호로 열기
O(log n)	약 20 번	사전 찾기
O(n)	100 만 번	한 명씩 확인
O(n log n)	약 2 천만 번	좋은 정렬
$O(n^2)$	1 조 번	모두가 서로 악수

같은 결과라도 방법에 따라 1 초 vs 1 시간 — 코딩 테스트의 핵심 주제 (7 장)

어떤 자료구조를 고를까

리스트

번호로 꺼내기

순위표

스택

마지막 것부터

뒤로 가기 · Ctrl+Z

큐

먼저 온 것부터

주문 대기열

해시

이름으로 빨리 찾기

회원 ID 찾기

트리

계층 · 정렬된 검색

폴더 · DB 인덱스

그래프

연결 · 경로

지하철 · 길 찾기

개발자의 기본 도구

1	**터미널**	글자로 명령하기	2	**VS Code**	코드 편집기
3	**Git · GitHub**	코드 타임머신	4	**패키지 관리자**	pip · npm 등
5	**디버거**	중단점 찍고 값 보기	6	**테스트**	자동으로 맞는지 확인

디버깅 순서 : 재현 → 범위 좁히기 → 가설 → 확인 → 고치고 테스트 추가 (8 장)

웹 = 요청과 응답

브라우저

요청 : GET /api/products →

← 응답 : 200 OK + JSON

서버 + DB

2xx

성공

3xx

이동

4xx

손님 쪽 문제 (404)

5xx

서버 쪽 문제 (500)

SQL: 데이터베이스에 묻는 언어

```
SELECT name, price FROM products
  WHERE price < 10000;
INSERT INTO products (name, price)
  VALUES ('주스', 4500);
UPDATE products SET price = 3500
  WHERE name = '쿠키';
```

CRUD

만들기 INSERT · 읽기 SELECT · 고치기 UPDATE · 지우기 DELETE

사고 방지 WHERE 빠지면 표 전체가 바뀜 · 손님 입력은 자리표시자 (?) 로 → 인젝션 차단

메모리는 누가 치우나

직접 치우기

C · C++

빠르고 정밀 ↔ 잊으면 누수 , 실수하면 보안 구멍

청소부 (GC)

자바 · 파이썬 · JS · Go · C#

편하고 안전 ↔ 청소할 때 잠깐 멈출 수 있음

소유권 규칙

러스트

빠르고 안전 ↔ 규칙이 엄격해 배우기 어려움

스택 = 책상 위 메모지 (자동 정리) · 힙 = 창고 (누군가 치워야 함)

동시성 : 빠르지만 함정이 있다

경쟁 상태

두 명이 재고 1개를 동시에 판매 → 재고 -1

해결 : 락 · DB 트랜잭션

교착 상태

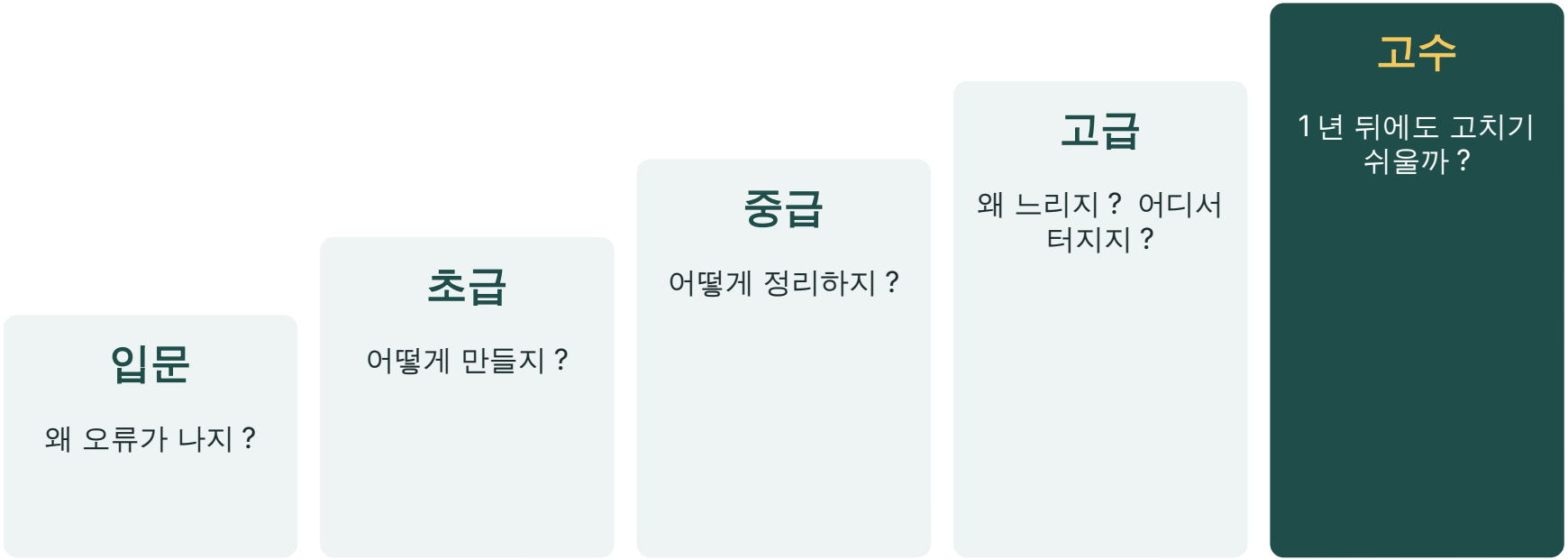
좁은 골목에서 서로 비켜 주길 기다림

해결 : 자원 잡는 순서 통일

성능 개선 순서 측정 → 알고리즘 · 자료구조 → 불필요한 반복 제거 → 캐시 → 더 빠른 언어 · 병렬

입문에서 고수까지 : 고민이 달라진다

고수 = 여러 사람이 오래 고쳐도 안 무너지는 시스템을 만드는 사람



고수들의 설계 상식

원칙

- SOLID — 한 가지 책임 , 갈아 끼우기 쉽게
- KISS — 단순하게
- YAGNI — 지금 필요 없는 건 만들지 않기
- DRY — 같은 지식을 두 곳에 두지 않기
- 코드 리뷰 · 테스트 · 리팩터링

큰 서비스의 부품

- 로드 밸런서 — 손님 나눠 보내기
- 서버 여러 대 (수평 확장)
- 캐시 · 메시지 큐
- DB 복제 · 샤딩
- 작은 팀은 단순한 한 덩어리로 시작

AI 코딩 도구 : 빠른 후배처럼 쓰기

잘 쓰는 법

- 오류 원인 · 코드 설명 요청
- 작게 나눠 맡기기
- 목표 · 제약 · 완료 기준을 구체적으로
- 테스트 작성 맡기고 직접 검토

위험한 쓰는 법

- 이해 못 한 코드 그대로 붙여 넣기
- 비밀번호 · 회사 비밀 입력
- 테스트 없이 배포
- 처음부터 다 맡기기

설계 판단 · 결과 검증 · 최종 책임은 사람에게 — 기초를 아는 사람이 AI 를 더 잘 쓴다

진로별 첫 언어

가고 싶은 길	첫 언어	이어서 배울 것
웹 프론트엔드	HTML/CSS + JS	타입스크립트 · React
백엔드	자바 / 파이썬 / TS / Go	SQL · API · 리눅스 · 도커
모바일 앱	코틀린 / 스위프트	Flutter · 앱 배포
게임	C# (유니티) / C++	수학 · 네트워크
데이터 · AI	파이썬 + SQL	pandas · PyTorch · AI API
임베디드 · 보안	C	운영체제 · 네트워크 · 어셈블리

실력은 강의 수가 아니라 완성해서 배포한 프로젝트 수로 쌓인다 (12 장)

작게 만들고 , 끝까지 완성하기

숫자 맞히기 게임 → 가계부 → 자기소개 웹페이지 → 할 일 앱 + 서버 → 오목

다음 할 일 : docs/coding/04-basics-1.md 를 열고 예제를 직접 쳐 보기